# Лабораторна робота № 2

*Тема: Винесення операційного сервісу з моноліту та інтеграційні тести*

## 1. Мета роботи

Навчитися виносити з монолітного застосунку **сервіс, який виконує головну дію вашої предметної області** (створює, змінює та скасовує операції користувача), віддати цьому сервісу **власну базу даних**, налаштувати його взаємодію з раніше створеним сервісом каталогу та — найважливіше — **довести тестами, що після винесення система працює точно так само, як працювала до нього**.

За результатами роботи ви маєте вміти:

1. визначати межу (шов) між частиною моноліту, яку виносять, і тією, що залишається;
2. переносити дані у власну БД сервісу без зв'язків між базами;
3. організовувати виклик сусіднього сервісу з таймаутом та ідемпотентністю;
4. перемикати маршрут на шлюзі зі старого моноліту на новий сервіс;
5. писати **інтеграційні тести** на ASP.NET Core API та використовувати їх як захист від регресій.

## 2. Що саме ви виносите (узагальнення для різних предметних областей)

Тема роботи сформульована як «винесення Booking-сервісу», але **Booking (бронювання) — це лише один з можливих варіантів**. У кожного з вас своя предметна область, тому однакової сутності «Booking» ні в кого бути не зобов'язано.

Те, що ви виносите, у методичці далі називається **операційний сервіс** — сервіс, який відповідає за *дію* користувача над об'єктами каталогу, а не за самі об'єкти. Каталог відповідає на питання «що в нас є», операційний сервіс — на питання «хто, що і коли з цим зробив».

| **Предметна область (приклади)** | **Операційний сервіс** | **Головна сутність** |
|---|---|---|
| Готель, коворкінг, спортзал, перукарня | Booking (бронювання) | Booking / Reservation |
| Інтернет-магазин, доставка їжі, аптека | Ordering (замовлення) | Order + OrderItem |
| Кінотеатр, театр, концерти, транспорт | Ticketing (продаж місць) | Ticket / Seat |
| Прокат авто, велосипедів, обладнання | Rental (оренда) | Rental / Agreement |
| Навчальна платформа, курси, секції | Enrollment (запис на курс) | Enrollment |
| Поліклініка, СТО, сервісний центр | Appointment (запис/заявка) | Appointment / Request |

| Предметна область (приклади) | Операційний сервіс | Головна сутність |
|---|---|---|
| Бібліотека, склад, музейний фонд | Loan / Issue (видача) | Loan / IssueAct |
| Логістика, пошта | Delivery (перевезення) | Shipment |

***А якщо у моїй темі нічого такого немає?***

Так буває: наприклад, предметна область — «моніторинг датчиків», «бібліотека шейдерів», «довідник законодавства». Тоді ви виносите **наступний за важливістю сервіс за вашою власною таблицею володіння даними з ЛР0** (ADR-0001). Правило вибору просте — виносимо той модуль, який:

- **змінює** стан системи (має операції створення/редагування/скасування), а не тільки читає;
- має **власний набір таблиць**, які більше нікому не потрібні на запис;
- **звертається** до сервісу каталогу (або до того сервісу, який ви вже винесли в ЛР1), тобто дає нам можливість потренувати міжсервісний виклик.

> **Важливо:** Який саме сервіс виносити — **вирішуєте ви самі**, узгоджувати ні з ким не треба. Але рішення має бути **обґрунтованим**: у ADR-0003 пишете одним абзацом, чому обрали саме його («виношу сервіс X, бо він володіє таблицями Y, змінює стан системи і звертається до Catalog за Z»). Оцінюється не вибір, а якість обґрунтування і відповідність критеріям з цього розділу.

Далі в тексті сервіс позначено як **Operations-сервіс**; у своєму коді називайте його за своєю предметною областю: `BookingService`, `OrderingService`, `TicketingService`, `RentalService` тощо. Назви `miniproject_booking`, `BookingDbContext`, порт 5003 у прикладах — це *шаблон*, замінюйте на свої.

## 3. Головне правило цієї роботи

> **НАЙВАЖЛИВІШЕ: Після винесення сервісу система повинна працювати точно так само, як працювала до винесення.** Ті самі маршрути, ті самі коди відповідей, ті самі поля у JSON, та сама бізнес-логіка, ті самі перевірки. Користувач (і клієнтський застосунок) не повинен помітити жодної різниці, окрім, можливо, незначної зміни часу відповіді.

Це не формальна фраза, а умова прийняття роботи. Найчастіші помилки студентів під час винесення — і кожна з них є **багом, якого до винесення не було**:

1. змінився формат відповіді (було `{ "id": 1 }`, стало `{ "Id": 1 }` або додалися зайві поля);
2. зник код `404` — сервіс почав повертати `500` або `200` з пустим тілом;
3. зникла перевірка вхідних даних, яка була в моноліті (наприклад, «дата початку не може бути в минулому»);
4. перестала працювати авторизація: ендпоінт, який раніше вимагав токен, став публічним;

5. замовлення/бронювання тепер можна створити на неіснуючий об’єкт каталогу, бо перевірку «забули» перенести;
6. дата стала приходити в іншому часовому поясі або в іншому форматі;
7. змінився порядок або нумерація: ідентифікатори почались з 1 і наклались на старі дані.

Щоб таких помилок не було, порядок дій **завжди** такий: *спочатку зафіксувати поведінку, потім змінювати код, потім порівняти*. Саме для цього в роботі є інтеграційні тести.

> **Регресія —** повернення дефекту або втрата функціональності, яка раніше працювала. «Регресійний тест» — тест, який ми тримаємо саме для того, щоб вчасно побачити регресію.
>
> ***Походження терміна:*** *від лат. regressus — «рух назад, повернення».*

> **Характеризаційний тест (characterization test, baseline test) —** тест, який пишуть **не** за документацією, а за **фактичною** поведінкою наявної системи: що код реально повертає сьогодні — те й записуємо як очікуване. Такі тести не доводять, що система правильна; вони доводять, що вона **не змінилася**.
>
> ***Походження терміна:*** *термін ввів Майкл Фізерс (Michael Feathers) у книжці «Working Effectively with Legacy Code», 2004.*

# 4. Методичні відомості

## 4.1. Де проходить шов

> **Шов (seam) —** місце в системі, де її можна «розрізати», змінивши поведінку однієї частини й не змінюючи код іншої. У нашому випадку шов — це межа між модулем, який ми виносимо, і рештою моноліту.
>
> ***Походження терміна:*** *англ. seam — «шов» (на тканині); термін також з книжки Фізерса.*

Практично шов шукають так: виписують усі таблиці й усі класи, яких торкається модуль, і ділять їх на три групи.

| Група | Приклад | Що робимо |
|---|---|---|
| Наші дані (пишемо й читаємо) | Bookings, OrderItems, Payments | Переносимо у власну БД сервісу |
| Чужі дані (тільки читаємо) | Rooms, Products, Movies, Users | Не переносимо. Питаємо власника по мережі або зберігаємо копію-знімок |
| Спільна технічна дрібниця | перелічення, довідник статусів | Дублюємо у себе — це дешевше, ніж спільна бібліотека |

> **Увага: Між базами різних сервісів не буває зовнішніх ключів.** Якщо у вашій таблиці бронювань був `FOREIGN KEY (RoomId) REFERENCES Rooms(Id)`, після винесення залишається просто колонка `RoomId` без FK, а перевірку «така кімната існує» виконує **виклик до Catalog-сервісу**.

## 4.2. Крок 0: зафіксувати поведінку ДО змін

Це єдиний крок, який не можна пропустити. Поки модуль ще живе в моноліті, ви знімаєте з нього «відпечаток»: набір запитів і **точних** відповідей.

Найпростіший спосіб — файл `baseline.http` (Visual Studio / Rider / VS Code підтримують такі файли) або скрипт на `curl`, який послідовно виконує сценарії й складає відповіді у файли:

```
# baseline.sh — запускаємо ПРОТИ МОНОЛІТУ, до будь-яких змін
BASE=http://localhost:5100
TOKEN=$(curl -s -X POST $BASE/api/auth/login \
        -H "Content-Type: application/json" \
        -d '{"email":"test@test.com","password":"Test123!"}' | jq -r .accessToken)

# 1. створення операції — зберігаємо і код статусу, і тіло
curl -s -o out/01-create.json -w "%{http_code}\n" -X POST $BASE/api/bookings \
     -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
     -d @requests/create-valid.json          > out/01-create.status

# 2. створення з неіснуючим об'єктом каталогу
# 3. створення з датою в минулому
# 4. ... решту сценаріїв допишіть самостійно (див. розділ 10)
```

Отримані файли **комітите в репозиторій** у теку `tests/baseline/`. Саме з ними ви потім порівнюєте відповіді нового сервісу. Якщо різниця є — або ви щось зламали, або різниця свідома й пояснена у файлі `docs/baseline-diff.md` (наприклад, змінився лише заголовок `Server`).

> **Увага:** Знімайте базову поведінку **не тільки для «щасливих» сценаріїв**. Найцінніше — поведінка на помилках: що саме повертає моноліт, коли дані невалідні, коли об'єкта немає, коли токен прострочений. Саме ці відповіді студенти найчастіше й гублять.

### 4.3. Власна база даних сервісу

Кожен сервіс має власну схему й власний `DbContext`. Для Operations-сервісу це означає окрему БД (наприклад `miniproject_booking`) і міграції, які живуть у його проєкті.

```
// Program.cs нового сервісу — фрагмент
var cs = builder.Configuration.GetConnectionString("OperationsDb")!;
builder.Services.AddDbContext<OperationsDbContext>(opt =>
    opt.UseMySql(cs, ServerVersion.AutoDetect(cs)));

// OperationsDbContext.cs — залишок конфігурації ваш
protected override void OnModelCreating(ModelBuilder b)
{
    // TODO: задайте точність для грошових полів, індекси для пошуку,
    //       перелічення статусу збережіть як рядок (HasConversion<string>()),
    //       і НЕ оголошуйте навігацію на сутності каталогу — їх тут немає.
}
```

#### *Перенесення наявних даних*

Дані зі старих таблиць моноліту треба перенести, інакше «система працює так само» не виконується: старі бронювання зникнуть. Варіанти — на ваш вибір, але вибір треба пояснити під час захисту роботи:

- **SQL-скрипт перенесення** (`INSERT INTO miniproject_booking.Bookings SELECT ... FROM miniproject.Bookings`) — найпростіше, працює, якщо обидві БД на одному сервері MySQL;
- **сідер у новому сервісі**, який під час першого запуску читає дані зі старої БД і записує у свою;
- **експорт у CSV/JSON і імпорт** — найгнучкіше, але найбільше ручної роботи.

**Увага:** Після перенесення обов’язково підніміть стартове значення автоінкременту в новій таблиці вище за максимальний старий Id. Інакше новий сервіс почне видавати ідентифікатори, які вже існують у відповідях старого API та в клієнтах.

### 4.4. Виклик сусіднього сервісу

Перевірка «об’єкт каталогу існує і доступний» тепер не SQL-запит, а мережевий виклик до Catalog-сервісу — за gRPC (швидше, контракт в `.proto`) або за HTTP через типізований `HttpClient`. Фрагмент для gRPC-клієнта:

```
// Program.cs
builder.Services.AddGrpcClient<CatalogService.CatalogServiceClient>(o =>
{
    o.Address = new Uri(builder.Configuration["Services:CatalogGrpc"]!);
});

// у обробнику створення операції
var deadline = DateTime.UtcNow.AddSeconds(2);   // таймаут обов’язковий!
try
{
    var item = await _catalog.GetItemAsync(new GetItemRequest { Id = dto.ItemId },
                                           deadline: deadline, cancellationToken: ct);
    // TODO: перевірте не лише факт існування, а й доступність/статус об’єкта
}
catch (RpcException ex) when (ex.StatusCode == StatusCode.NotFound)
{
    // TODO: яку саме відповідь віддавав моноліт у цьому випадку? Її і віддайте.
}
catch (RpcException ex) when (ex.StatusCode == StatusCode.DeadlineExceeded
                           || ex.StatusCode == StatusCode.Unavailable)
{
    // TODO: сервіс каталогу недоступний. 503 + Retry-After? чи інша стратегія?
}
```

**Дедлайн (deadline) у gRPC —** момент часу, до якого виклик має завершитися; якщо не завершився — клієнт отримує помилку `DeadlineExceeded` і перестає чекати. На відміну від звичайного таймауту дедлайн передається по мережі й «розповзається» по всьому ланцюжку викликів.

**Ідемпотентність —** властивість операції давати **той самий результат** при повторному виконанні з тими самими даними. Повторне `GET` ідемпотентне завжди; повторне `POST /bookings` — ні, якщо не подбати про це окремо. Практичний спосіб: клієнт надсилає заголовок `Idempotency-Key` з унікальним значенням, сервіс запам’ятовує ключ і на повторний запит з тим самим ключем повертає раніше створений результат, а не створює другу операцію.

***Походження терміна:*** *від лат. idem — «той самий» і potens — «здатний»; термін із математики (ідемпотентний елемент: $x \cdot x = x$), у веб-API поширився з HTTP-специфікацій 1990-х.*

**Кореляційний ідентифікатор (correlation id) —** рядок, який клієнт або шлюз генерує для **одного** запиту й який далі передається в усі сервіси, задіяні в його обробці. Дає змогу зібрати в логах повну історію одного запиту, що пройшов через 3–4 процеси.

### 4.5. Перемикання маршруту на шлюзі

Поки сервіс не готовий, шлюз шле запити в моноліт (так було в ЛР0). Коли новий сервіс пройшов тести — змінюєте **лише конфігурацію шлюзу**, не чіпаючи клієнтів:

```
// ocelot.json — було (запити йшли в моноліт)
{
  "UpstreamPathTemplate":   "/api/bookings/{everything}",
  "DownstreamPathTemplate": "/api/bookings/{everything}",
  "DownstreamHostAndPorts": [ { "Host": "localhost", "Port": 5100 } ]
}

// ocelot.json — стало (запити йдуть у новий сервіс)
{
  "UpstreamPathTemplate":   "/api/bookings/{everything}",
  "DownstreamPathTemplate": "/api/bookings/{everything}",
  "DownstreamHostAndPorts": [ { "Host": "localhost", "Port": 5003 } ]
}
```

Те, що для перемикання достатньо змінити один рядок конфігурації, і є практична користь від шлюзу. Старий код моноліту **не видаляємо одразу** — залишаємо на час перевірки; під час захисту пояснюєте, коли і як його прибрали або чому залишили.

### 4.6. Інтеграційні тести

**Модульний (unit) тест —** перевіряє один невеликий фрагмент коду (метод, клас) **в ізоляції**, без бази даних, мережі й файлів. Швидкий, але нічого не говорить про систему в цілому.

**Інтеграційний тест —** перевіряє, що **кілька складових працюють разом**: контролер + конвеєр ASP.NET Core + валідація + EF Core + реальна (тестова) база. Для ASP.NET Core такий тест піднімає застосунок у пам’яті й надсилає до нього справжні HTTP-запити.

***Походження терміна:*** *від лат. integratio — «відновлення цілого»; у тестуванні термін закріпився в 1970-х разом із моделлю «unit → integration → system».*

**WebApplicationFactory —** клас з пакета `Microsoft.AspNetCore.Mvc.Testing`, який запускає ваш застосунок у тестовому процесі та видає `HttpClient`, підключений до нього напряму — без реального сокета й без `dotnet run`. Дає змогу в тесті підмінити будь-яку залежність (наприклад, БД або gRPC-клієнта каталогу).

**Стаб (stub) / фейк (fake) —** підставна реалізація залежності, яка повертає заздалегідь відомі відповіді. У цій роботі стаб потрібен для Catalog-сервісу: тест не має підіймати ще один процес, щоб перевірити логіку операцій.

Каркас тестового проєкту (створіть його як окремий проєкт `tests/Operations.IntegrationTests`):

```
// пакети: Microsoft.AspNetCore.Mvc.Testing, xunit, xunit.runner.visualstudio,
//         FluentAssertions (необов'язково), Testcontainers.MySql (необов'язково)

public class ApiFactory : WebApplicationFactory<Program>
{
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Testing");
        builder.ConfigureServices(services =>
        {
            // TODO 1: приберіть справжній DbContext і підключіть ТЕСТОВУ базу
            //         (окрема БД miniproject_booking_test або контейнер)
            // TODO 2: підмініть клієнта каталогу стабом із відомими відповідями
            // TODO 3: подумайте, як у тесті отримати валідний JWT
        });
    }
}

public class CreateOperationTests : IClassFixture<ApiFactory>
{
    private readonly HttpClient _client;
    public CreateOperationTests(ApiFactory f) => _client = f.CreateClient();

    [Fact]
    public async Task Create_WithValidData_Returns201AndLocation()
    {
        // TODO: порівняйте і код статусу, і поля тіла з baseline-файлом
    }
}
```

**Увага:** Тестова база **не може бути тією самою, що й робоча**. Тест має право видаляти й перестворювати схему, тому перед запуском переконайтеся, що в рядку підключення саме тестова БД (найпростіше — `appsettings.Testing.json`).

Кожен тест має бути **незалежним**: він сам створює потрібні йому дані й не спирається на те, що залишив попередній. Два прийоми: перестворення схеми перед класом тестів (`EnsureDeleted` + `Migrate`) або виконання кожного тесту в транзакції, яка відкочується.

# 5. Підготовка середовища

| Складова | Порт | База даних |
|---|---|---|
| API Gateway (Ocelot) | 5000 | — |
| Auth-сервіс | 5001 | miniproject_auth |
| Catalog-сервіс | 5002 / gRPC 5102 | miniproject_catalog |
| **Operations-сервіс (ця ЛР)** | **5003 / gRPC 5103** | **miniproject_booking** (своя назва) |
| Залишок моноліту | 5100 | miniproject (стара БД) |
| Тестова БД для інтеграційних тестів | — | miniproject_booking_test |

Структура репозиторію після роботи:

```
MiniProject/
  docs/adr/0001-decomposition.md        # з ЛР0
  docs/adr/0003-operations-extraction.md # НОВИЙ: чому і як виносили
  docs/diagrams/                        # оновлені діаграми
  shared/protos/catalog.proto           # з ЛР1
  src/Gateway/
  src/AuthService/
  src/CatalogService/
  src/OperationsService/                # НОВИЙ (назвіть за своєю областю)
  src/Monolith/
  tests/baseline/                       # НОВЕ: знімок поведінки до змін
  tests/Operations.IntegrationTests/    # НОВЕ
  docker-compose.yml  .env.example
```

**Увага:** Секрети (рядки підключення з паролями, `Jwt:Key`) у репозиторій не потрапляють **ніколи** — лише `user-secrets`, змінні середовища та `.env` (у `.gitignore`), а в репозиторії — `.env.example` з порожніми значеннями. Знайдений у коміті ключ — підстава повернути роботу на доопрацювання; такий ключ вважається скомпрометованим і його треба замінити, а не просто видалити рядок.

## 6. Модель даних

Нижче — **мінімальний** склад головної сутності. Типи, обмеження, індекси та додаткові поля визначаєте самостійно, виходячи зі своєї предметної області та з того, що вже було в моноліті.

| Поле | Призначення | Зауваження |
|---|---|---|
| Id | Первинний ключ | Тип узгодьте зі старою таблицею |
| UserId | Хто створив операцію | Беремо з JWT, а не з тіла запиту |
| ItemId | Об’єкт каталогу | Без FK — каталог в іншій БД |
| ItemSnapshot | Назва/ціна на момент операції | Денормалізована копія: історія не має «пливти» за каталогом |
| Status | Стан операції | Перелічення; перелік станів і дозволені переходи — ваші |
| CreatedAt / UpdatedAt | Час створення та зміни | Завжди UTC |
| IdempotencyKey | Ключ захисту від дублів | Унікальний індекс |
| *(ваші поля)* | Дати, кількість, сума, місце, коментар… | Залежить від області |

**Увага:** Поле `ItemSnapshot` — місце, де багато хто помиляється. Якщо ви зберігаєте лише `ItemId`, то після зміни ціни в каталозі старе замовлення «перепишеться» новою ціною. У моноліті з одною БД це теж було проблемою, але тут вона стає очевидною. Збережіть копію назви та ціни на момент операції.

## 7. Контракт API

Маршрути мають **залишитися тими самими**, що були в моноліті (саме тому шлюз перемикається прозоро). Нижче — типовий набір; свій зафіксуйте в документації OpenAPI свого сервісу.

| Метод і маршрут | Доступ | Призначення |
|---|---|---|
| GET /api/operations | авторизований | Список **своїх** операцій; пагінація, фільтр за статусом |
| GET /api/operations/{id} | власник або адмін | Одна операція; 404, якщо чужа або немає |
| POST /api/operations | авторизований | Створити; перевірка каталогу; `Idempotency-Key` |
| PUT /api/operations/{id} | власник | Змінити (якщо ваша область це допускає) |
| POST /api/operations/{id}/cancel | власник або адмін | Скасувати — зміна статусу, не видалення |
| GET /api/operations/all | адмін | Усі операції з фільтрами |
| GET /health | публічний | Стан сервісу та доступність його БД |

> **Увага:** Назви маршрутів у таблиці узагальнені. У себе лишаєте свої: `/api/bookings`, `/api/orders`, `/api/tickets`, `/api/enrollments`. **Перейменовувати наявні маршрути в цій роботі заборонено** — це зміна поведінки.

## 8. Завдання на реалізацію

Послідовність обов’язкова: пункти 1–2 виконуються **до** будь-яких змін у коді.

1. **Визначити, що виносите.** Складіть таблицю «наші дані / чужі дані» (розділ 4.1), самостійно оберіть сервіс і запишіть рішення разом з обґрунтуванням як ADR-0003.
2. **Зняти базову поведінку.** Підготуйте не менше 8 сценаріїв (розділ 10), виконайте їх проти моноліту й збережіть відповіді в `tests/baseline/`.
3. **Створити проєкт сервісу** з власним `DbContext`, міграціями й окремою БД. Ваше завдання: самостійно визначити склад сутностей, обмеження й індекси.
4. **Перенести логіку**, а не скопіювати контролер. Перевірки, які були в моноліті, мають бути перенесені **усі**; перевірки, які спиралися на SQL до чужих таблиць, замінюються на виклик сусіднього сервісу.
5. **Перенести дані** зі старих таблиць і виправити автоінкремент.
6. **Реалізувати виклик до Catalog-сервісу** з таймаутом і коректною обробкою трьох випадків: об’єкт не знайдено, сервіс недоступний, відповідь надто довга.
7. **Реалізувати ідемпотентність створення** через заголовок `Idempotency-Key`.
8. **Написати інтеграційні тести** — не менше 8 сценаріїв з розділу 10, усі зелені.
9. **Порівняти з базовою поведінкою.** Створіть у репозиторії файл `docs/baseline-diff.md` з таблицею «сценарій / було / стало / чи є різниця / чому». Рядків з різницею в ідеалі немає; кожна наявна різниця має пояснення.

10. **Перемкнути маршрут на шлюзі** й повторити ті самі сценарії вже через шлюз (порт 5000): результат має бути такий самий.
11. **Оновити документацію**: Swagger/OpenAPI нового сервісу, ADR-0003, діаграму компонентів з ЛР0 (додати новий сервіс і його БД).

## 9. Обов'язкові вимоги до сервісу

Ці вимоги наскрізні: вони діють для **кожного** сервісу в усіх роботах курсу.

1. **Валідація вхідних даних** на рівні моделі та бізнес-правил; помилки — у форматі `ProblemDetails` (RFC 9457) з переліком полів.
2. **Повне документування в OpenAPI**: описи ендпоінтів і полів, приклади запитів і відповідей, усі можливі коди статусів, схема авторизації Bearer. Порожній Swagger «за замовчуванням» не приймається.
3. **Можливість розширення**: логіку винесено в сервіси/обробники, залежності — через інтерфейси; додавання нового статусу або нового правила не повинно вимагати правок у контролері.
4. **Авторизація**: кожен ендпоінт має свідомо визначений доступ; `UserId` береться з токена, а не з тіла запиту. Доступ до чужої операції — 404 (а не 403, щоб не розкривати існування).
5. **Жодних секретів у репозиторії**; конфігурація — за ієрархією `appsettings` → `appsettings.{Environment}` → user-secrets → змінні середовища.
6. **Єдина обробка помилок** (`IExceptionHandler`); клієнт ніколи не бачить stack trace.
7. **Логування** з рівнями та кореляційним ідентифікатором; у логах немає паролів, токенів і персональних даних.
8. **Health check** `/health`, що перевіряє і БД, і доступність сусіднього сервісу (`/health/ready` окремо, якщо робите).
9. **Міграції EF Core** у репозиторії; база створюється з нуля однією командою.
10. **Асинхронність**: усі операції з БД і мережею — `async/await` з передаванням `CancellationToken`.

## 10. Сценарії, які треба перевірити (мінімум 8)

Ці сценарії ви знімаєте з моноліту (розділ 4.2) **і** реалізуєте як інтеграційні тести (розділ 4.6). Формулювання узагальнені — підставте свою предметну область.

| № | Сценарій | Що перевіряємо |
|---|---|---|
| 1 | Створення з коректними даними | 201, тіло з Id, заголовок Location, запис у БД |
| 2 | Створення без токена | 401 — і саме так, як це робив моноліт |
| 3 | Створення з неіснуючим ItemId | Код і текст помилки незмінні (зазвичай 400 або 404) |
| 4 | Створення з порушенням бізнес-правила | Правило збереглося (дата в минулому, кількість ≤ 0, вже зайнято) |

| № | Сценарій | Що перевіряємо |
|---|---|---|
| 5 | Повтор запиту з тим самим Idempotency-Key | Друга операція **не** створюється, відповідь та сама |
| 6 | Отримання своєї операції | 200 і той самий набір полів, що й раніше |
| 7 | Спроба отримати чужу операцію | 404, чужі дані не розкриваються |
| 8 | Скасування операції | Статус змінився, повторне скасування оброблено коректно |
| 9 | Каталог недоступний (стаб кидає помилку) | Сервіс не «падає»: 503 або інша свідома відповідь |
| 10 | Список з пагінацією й фільтром | Ті самі поля, та сама сортування й кількість |

**Увага:** Сценарії 1–8 — обов'язкові, 9–10 дають перевагу під час оцінювання. Кожен тест має перевіряти **і код статусу, і структуру тіла**: тест, що перевіряє лише `IsSuccessStatusCode`, зараховується як половина.

## 11. Додаткові завдання (плюшки)

| Плюшка | Що зробити | Бонус |
|---|---|---|
| Dockerfile + docker-compose | Сервіс, його БД і сусіди піднімаються однією командою `docker compose up` | +3 бали |
| Testcontainers | Інтеграційні тести піднімають MySQL у контейнері, а не вимагають локальну БД | перевага |
| Порівняння baseline автоматично | Скрипт, що сам порівнює відповіді до/після й друкує різницю | перевага |
| Паралельний режим (shadow read) | Шлюз шле той самий GET і в моноліт, і в новий сервіс, порівнює відповіді й пише розбіжності в лог | перевага |
| Повторні спроби з backoff | Polly: 3 спроби з наростаючою паузою при `Unavailable` | перевага |
| Фонове прибирання | BackgroundService, що переводить прострочені операції у відповідний статус | перевага |

## 12. Порядок перевірки

1. Показуєте `tests/baseline/` і дату його створення (коміт **до** змін у коді).
2. Піднімаєте всі сервіси; показуєте, що БД у нового сервіса своя й у ній є перенесені старі записи.
3. Запускаєте `dotnet test` — усі інтеграційні тести зелені. Викладач може попросити зламати один рядок логіки та показати, що тест став червоним.
4. Виконуєте 3–4 сценарії вручну **через шлюз** (порт 5000); відповіді порівнюються з baseline.

5. Зупиняєте Catalog-сервіс і показуєте, що Operations-сервіс не «падає», а віддає свідому відповідь.
6. Показуєте Swagger: описи, приклади, коди відповідей, авторизація.
7. Відповідаєте на контрольні питання.

# 13. Критерії оцінювання

| **Що оцінюється** | **Бали** |
|---|---|
| Самостійний вибір сервісу та його обґрунтування: таблиця володіння даними, ADR-0003 | 1 |
| Знімок базової поведінки до змін (≥ 8 сценаріїв, у репозиторії) | 1 |
| Сервіс з власною БД, міграції, перенесені дані | 2 |
| Перенесена логіка: усі старі перевірки на місці, маршрути не змінені | 1,5 |
| Виклик сусіднього сервісу: таймаут і три випадки обробки помилок | 1 |
| Ідемпотентність створення | 0,5 |
| Інтеграційні тести (≥ 8, перевіряють статус і тіло, незалежні) | 2 |
| Файл `docs/baseline-diff.md`: порівняння «було / стало» без необґрунтованих розбіжностей | 0,5 |
| Перемкнений маршрут на шлюзі + оновлені Swagger і діаграми | 0,5 |
| **Разом** | **10** |
| Бонус: Dockerfile і docker-compose | **+3** |

***Знижується оцінка за:***

- будь-яку зміну поведінки, не описану й не обґрунтовану в `docs/baseline-diff.md` (інші коди статусів, інші поля, зникла перевірка) — **до 3 балів**;
- зовнішні ключі або SQL-запити між базами різних сервісів — **2 бали**;
- baseline, знятий **після** змін у коді (видно за історією комітів) — **1 бал**;
- тести, які залежать від порядку запуску або від даних робочої БД — **1 бал**;
- секрети в репозиторії — **робота повертається на доопрацювання**;
- `UserId` з тіла запиту замість токена — **1 бал**;
- порожній або неповний Swagger — **1 бал**;
- синхронні виклики до БД/мережі, відсутність `CancellationToken` — **0,5 бала**.

# 14. Контрольні питання

1. Що таке шов (seam) і за якими ознаками ви його шукали у своєму моноліті?
2. Чому між базами різних сервісів не роблять зовнішніх ключів? Що використовують замість них?
3. Що таке характеризаційний тест і чим він відрізняється від звичайного?

4. Чим інтеграційний тест відрізняється від модульного і від e2e-тесту?
5. Для чого потрібен `WebApplicationFactory` і що саме ви в ньому підмінили?
6. Що таке ідемпотентність? Наведіть HTTP-метод, ідемпотентний за специфікацією, і метод, який таким не є.
7. Навіщо виклику до сусіднього сервісу потрібен дедлайн? Що станеться без нього, якщо сусід «завис»?
8. Чому в операції зберігають копію назви та ціни об’єкта каталогу, а не лише його Id?
9. Як ви перенесли старі дані й чому підняли автоінкремент?
10. Що таке Strangler Fig і на якому його кроці ви зараз перебуваєте?
11. Як ви дізналися б, що винесення зламало поведінку, якби не було baseline-файлів?